

Programmi C

Compilazione e Esecuzione

- Compilatore GCC -

- Compilatore standard del **progetto GNU**
- Supporta diversi linguaggi (C, C++, Fortran, Ada, Go, ecc.)
 - in SO tipicamente si usa per il C
- È uno strumento essenziale per lo sviluppo su Linux/Unix,
 - gran parte del sistema operativo e dei tool è scritto in C

Traduzione da:

codice sorgente in linguaggio ad alto livello

a:

linguaggio macchina eseguibile dal processore

- Compilatore GCC -

- Compilatore standard del **progetto GNU**
- Supporta diversi linguaggi (C, C++, Fortran, Ada, Go, ecc.)
 - in SO tipicamente si usa per il C
- È uno strumento essenziale per lo sviluppo su Linux/Unix,
 - gran parte del sistema operativo e dei tool è scritto in C

Per installare gcc

```
sudo apt update  
sudo apt install build-essential
```

Fasi della Compilazione

Il compilatore GNU C in ambiente Unix, per generare il file eseguibile dal file sorgente, passa attraverso le seguenti fasi:

- Fase 1) **Preprocessore**
 - Il preprocessore legge un sorgente C e produce in output un altro sorgente C, dopo avere **espanso in linea le macro** (`#define MAX 100`) e **incluso i file** (`#include <nomefile>`)
- Fase 2) **Compilazione**
 - Traduzione del codice sorgente ricevuto dal preprocessore in **codice assembly**
- Fase 3) **Assembler** (creazione del file oggetto)
 - Crea il **codice oggetto** (file con il suffisso `.o`)
- Fase 4) **Linker** (creazione del file eseguibile)
 - Unisce le funzioni definite in altri file sorgenti o definite in librerie, con la funzione `main()` per creare il **file eseguibile**

- Il compilatore GNU **gcc** in ambiente Unix -

- Compilazione -

Per compilare in ambiente Unix, un programma C scritto nel file **source.c** si utilizza un compilatore GNU per sorgenti C, che viene invocato con il comando **gcc**:

Compilazione:

\$ **gcc** source.c  **a.out** (eseguibile prodotto, formato ELF)

Opzioni comando gcc:

- flag “**-o**”: \$ gcc **-o eseguibile.out sorgente.c**
- flag “**-v**”: per sapere la versione del compilatore
- flag “**-E**”: per invocare solo il preprocessore
- flag “**-c**”: per creare il codice oggetto senza chiamare il linker
- flag “**-O**”: ottimizzazione: O0, O1, O2 (standard), Os, Ofast
- flag “**-Wall**”: per aumentare il livello dei messaggi prodotti in fase di compilazione
- \$ **man** gcc (manuale in linea)

- Il messaggi del compilatore -

Messaggi del Compilatore

- Il compilatore invia dei messaggi all'operatore:
 - **Messaggi di avvertimento** (warning messages)
 - I messaggi di avvertimento indicano la presenza di parti di codice presumibilmente mal scritte o di problemi che potrebbero avvenire in seguito, durante l'esecuzione del programma. I messaggi di avvertimento non interrompono comunque la compilazione.
 - **Messaggi d'errore** (error messages)
 - I messaggi di errore invece indicano qualcosa che deve essere necessariamente corretto e causano l'interruzione della compilazione.
- Per **inibire** tutti i messaggi di **warning** usare l'opzione **-w**
 - `$ gcc -w -o eseguibile.out source.c`
- Per **aumentare** i **warning** usare l'opzione **-Wall** e **-Wextra**
 - `$ gcc -Wall -Wextra -o eseguibile.out source.c`

Compilare con opzione debug

- Per effettuare il debug di un programma, utilizzate sempre l'opzione **-g**:
 - `$gcc -Wall -g -o eseguibile.out source.c`
 - `$gcc -g -O0 -o eseguibile.out source.c` (O0 no ottimizzazione)
- L'opzione **-g** fa sì che il programma eseguibile contenga informazioni supplementari che permettono al debugger di collegare le istruzioni in linguaggio macchina che si trovano nell'eseguibile alle righe del codice corrispondenti nei sorgenti C
 - `gdb ./eseguibile`
 - `(gdb) break main` # imposta breakpoint all'inizio di main
 - `(gdb) run` # lancia il programma
 - `(gdb) list` # mostra le righe di codice
 - `(gdb) next` # esegui la prossima istruzione (step over)
 - `(gdb) step` # entra dentro la funzione chiamata
 - `(gdb) print "done"` # stampa qualcosa
 - `(gdb) continue` # riprendi fino alla fine
 - `(gdb) quit` # esci

Compilare con opzione debug

- Per effettuare il debug di un programma, utilizzate sempre l'opzione **-g**:
 - `$gcc -Wall -g -o eseguibile.out source.c`
 - `$gcc -g -O0 -o eseguibile.out source.c` (O0 no ottimizzazione)
- L'opzione **-g** fa sì che il programma eseguibile contenga informazioni supplementari che permettono al debugger di collegare le istruzioni in linguaggio macchina che si trovano nell'eseguibile alle righe del codice corrispondenti nei sorgenti C
 - `print x` → valore di una variabile
 - `info locals` → tutte le locali della funzione
 - `display x` → aggiorna sempre x automaticamente
 - `set var x = ...` → cambi il valore

Compilare con opzione debug

- Per effettuare il debug di un programma, utilizzate sempre l'opzione **-g**:
 - `$gcc -Wall -g -o eseguibile.out source.c`
 - `$gcc -g -O0 -o eseguibile.out source.c`

Es.

```
int main() {  
    int a = 5, b = 7;  
    int c = a + b;  
    return 0;  
}
```

```
(gdb) break main  
(gdb) run  
(gdb) next  
(gdb) info locals # mostra a=5  
(gdb) next  
(gdb) p b        # mostra b=7  
(gdb) next  
(gdb) p c        # mostra c=12
```

Funzione Principale C

Funzione principale in C

```
int main(void) {  
    // nessun argomento  
    return 0;  
}
```

Oppure

```
int main(int argc, char *argv[]) {  
    // argc = numero di argomenti (incluso il nome del programma)  
    // argv = array di stringhe con gli argomenti  
    return 0;  
}
```

Valore di ritorno:

- 0 = successo
- diverso da 0 = errore

Funzione Principale C

Funzione principale in C e argomenti

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]) {  
    printf("Numero argomenti: %d\n", argc);  
    for (int i = 0; i < argc; i++) {  
        printf("argv[%d] = %s\n", i, argv[i]);  
    }  
    return 0;  
}
```

Compilazione

```
gcc -o stampaArgomenti stampaArgomenti.c
```

Lancio

```
./stampaArgomenti uno due tre
```

```
Numero argomenti: 4  
argv[0] = ./prog  
argv[1] = uno  
argv[2] = due  
argv[3] = tre
```

Comandi Shell in C

Lancio di comandi shell da C

```
#include <stdlib.h>
```

```
int main() {  
    system("ls -l");  
    return 0;  
}
```

Qui system è funzione di libreria
lancia /bin/sh -c “comando”

Per lanciare bash
system(“bash -c ls -l”)

```
#include <stdlib.h>
```

```
int main(void) {  
    return system("bash -c 'ls -l'");  
}
```

Chiamata di Sistema in C

Chiamata di sistema di basso livello

```
#include <unistd.h>
#include <sys/syscall.h>
```

```
int main(void) {
    const char msg[] = "Hello, via syscall()\n";
    syscall(SYS_write, 1, msg, sizeof(msg) - 1); // write(1, msg, len)
    return 0;
}
```

Chiamata di Sistema

- Parametri inseriti nello stack dal programma e estratti dallo stack dal SO
- Programma in C invoca una chiamata di sistema:
 - Invocata da una procedura con il nome della sys call *read(fd, buffer, nbytes)*
 - Restituisce numero byte o -1 nel var globale errno

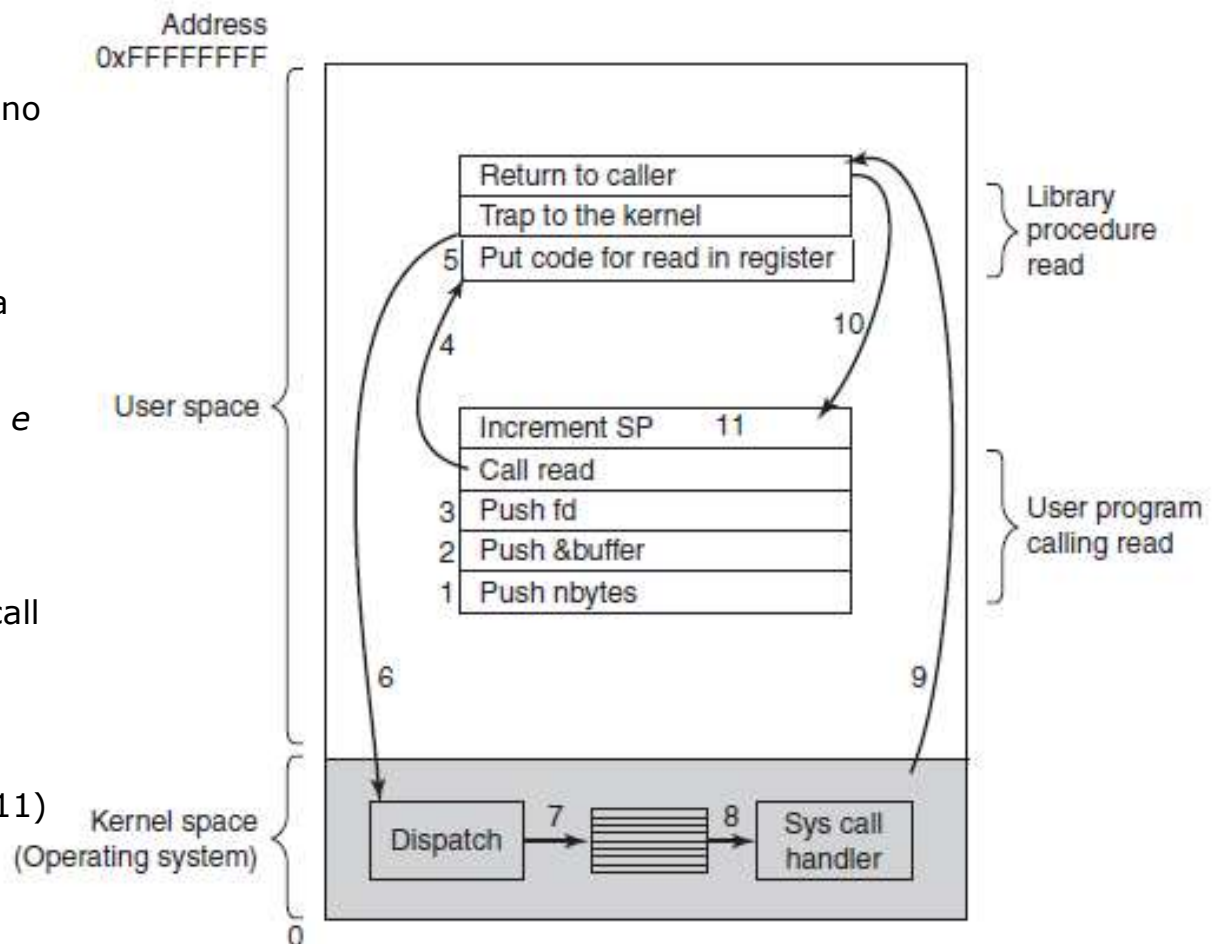
Il programma chiamante predispone la chiamata mettendo prima i parametri in uno stack (passi 1-3) – il buffer passato per riferimento

La procedura chiama il read (passo 4) mettendo il codice della chiamata a sistema di read in un registro atteso dal Kernel

Quindi manda una istruzione *Trap* al kernel e passa il controllo al kernel in kernel mode (passi 5-6)

Il kernel trova il codice della chiamata e mediante una tabella trova l'handler della call (passi 7 e 8) finita l'esecuzione passa il risultato al chiamante (passo 9)

Il chiamante riprende il controllo in user mode (passo 10) e pulisce lo stack (passo 11) incrementando lo Stack Pointer (SP)

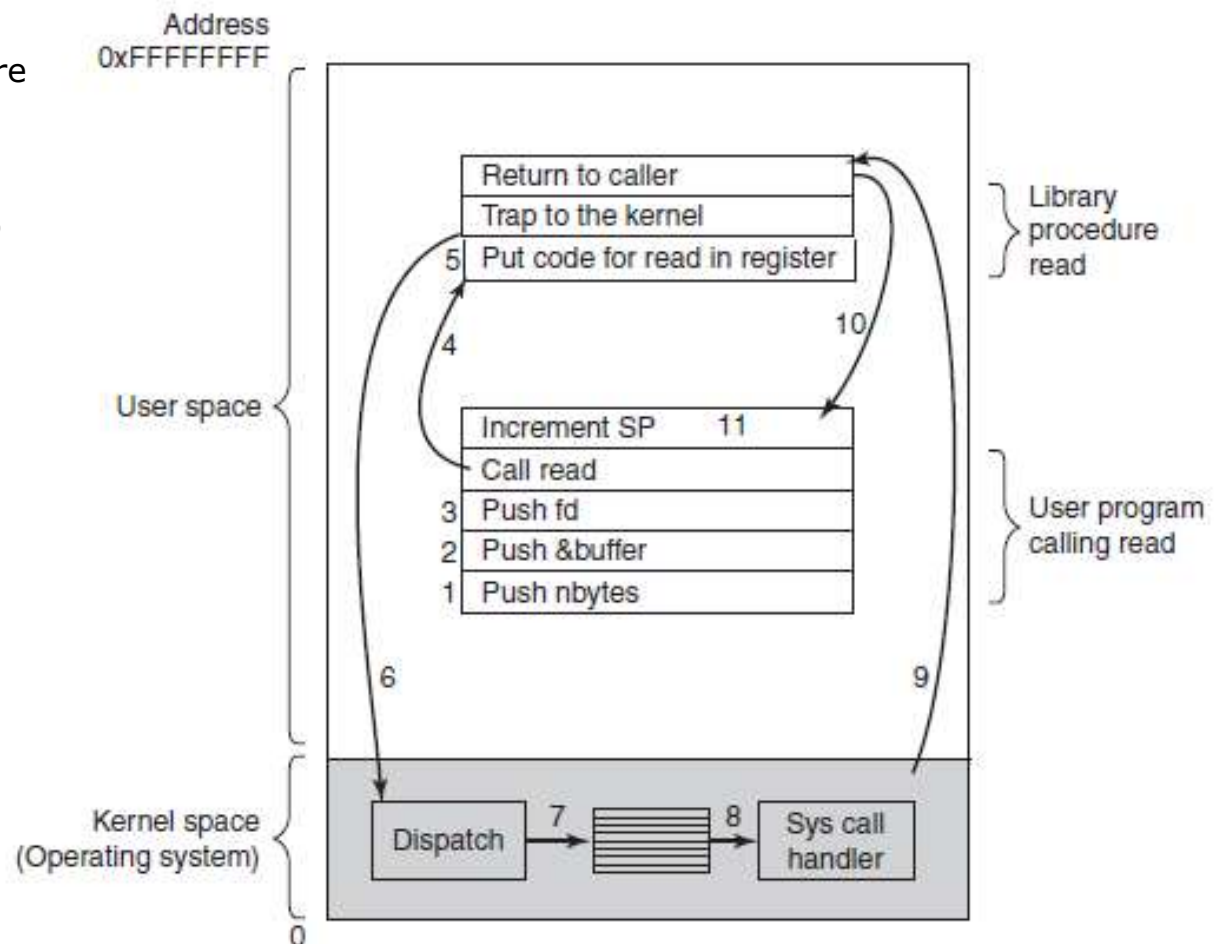


Chiamata di Sistema

- Parametri inseriti nello stack dal programma e estratti dallo stack dal SO
- Programma in C invoca una chiamata di sistema:
 - Invocata da una procedura con il nome della sys call *read(fd, buffer, nbytes)*
 - Restituisce numero byte o -1 nel var globale errno

Le procedure POSIX non si mappano sempre uno a uno con le system call offerte dal SO

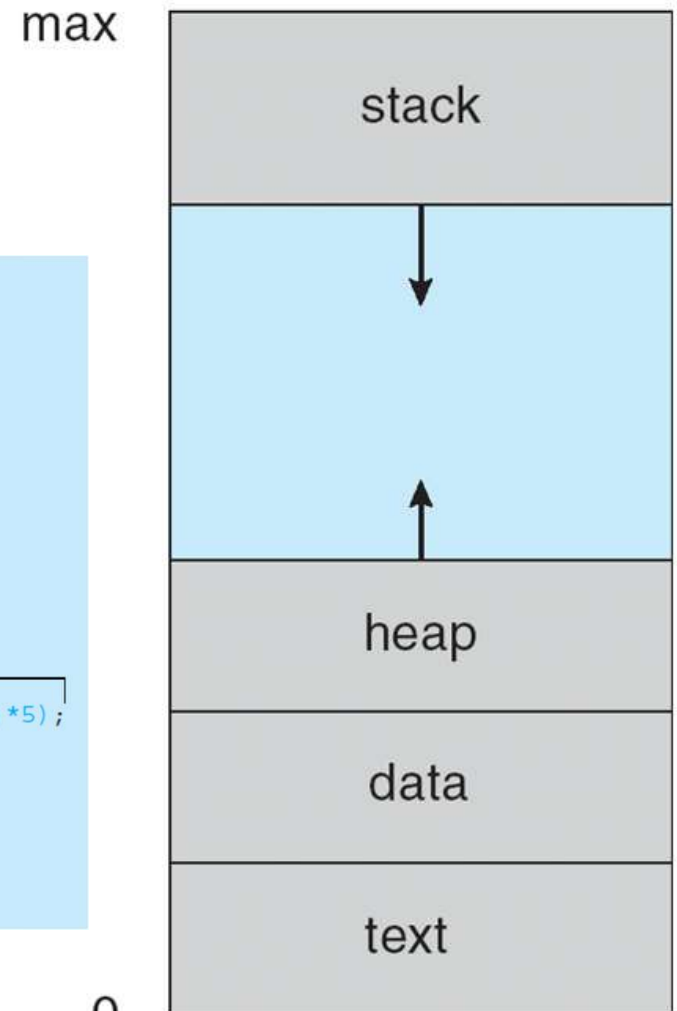
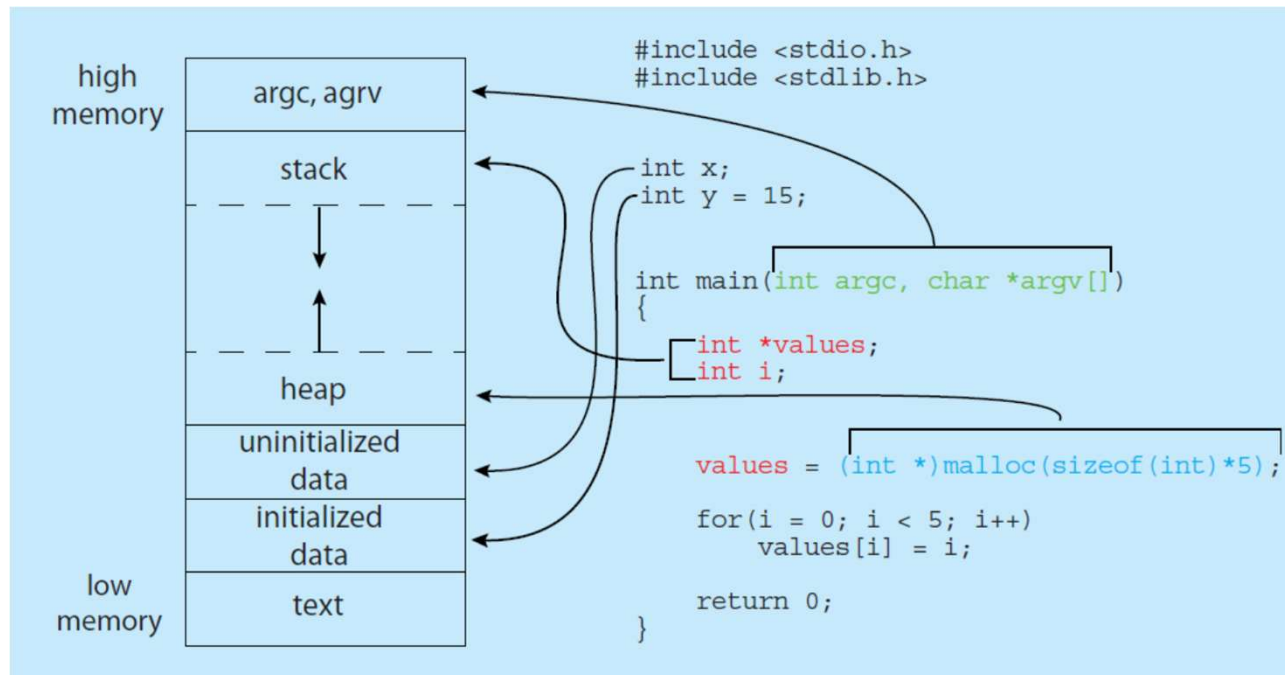
Nella maggior parte dei casi chiamano effettivamente le sys call. Alcune eseguono operazioni senza fare il TRAP del kernel



- Esecuzione e Caricamento -

Processo Allocato in Memoria

- **Processo** – è un programma in esecuzione:
- Layout di memoria:
 - Text e Data dimensione fissata



Non initialized data anche indicate come block started by symbol (bss)

Esecuzione e Caricamento

- Esecuzione Kernel invocate da `exec()`
 - Verifica che il processo chiamante ha i permessi
 - Kernel invoca un **loader** per iniziare l'esecuzione del programma
- Non una sola routine di loading
 - Linux mantiene una tabella di funzioni per il caricamento di programmi
 - Permette di supportare diversi formati binari (**ELF** e **a.out**)
 - **a.out** (Assembler output)
 - **ELF** (Executable and Linkable Format)
 - Linux ha usato a.out fino al Kernel 2.0, per poi passare a ELF
 - a.out deprecato nella versione 5.1

Esecuzione e Caricamento

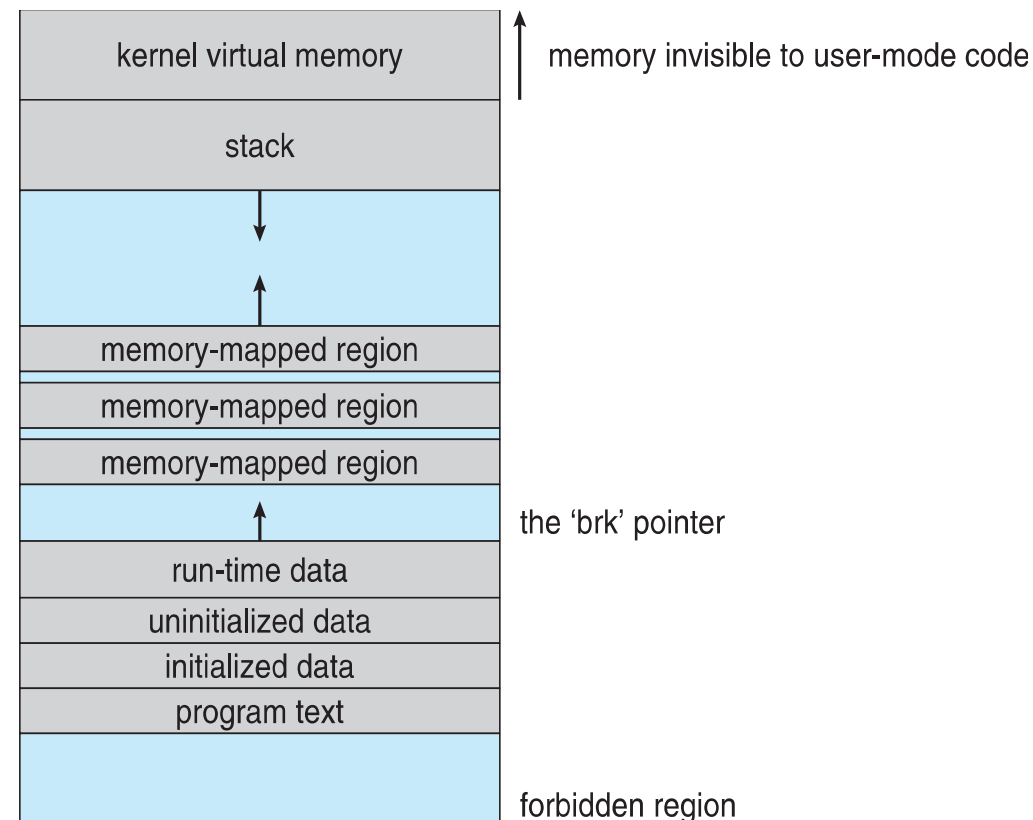
Mapping di Programmi in Memoria

- Inizialmente pagine di binary-file sono mappate in memoria
- Un file binario ELF consiste in un **header** e diverse **sezioni**
- L'**ELF loader** legge l'ELF header e la **Program Header Table**, e mappa i **segmenti LOAD** del file in regioni della memoria virtuale (text, data, bss, ecc.)

Le regioni caricate dal loader da ELF sono:

- **Program text** (codice, read+exec)
- **Data** (variabili inizializzate, read+write)
- **BSS** (variabili non inizializzate, azzerate in RAM)
- Lo **stack** viene creato dal kernel (in alto)
- L'**heap** è una regione variabile che cresce dinamicamente (brk, mmap); brk, marca la fine del segmento dati e inizio dell'heap

Fatto il mapping, il loader inizializza il program-counter del processo con lo starting point nell'ELF header e il processo può essere scheduled



Esecuzione e Caricamento

- Mapping di Programmi in Memoria
 - Inizialmente pagine di binary-file sono mappate in memoria
 - Un file binario ELF consiste in un **header** e diverse **sezioni**
 - L'**ELF loader** legge l'ELF header e la **Program Header Table**, e mappa i **segmenti LOAD** del file in regioni della memoria virtuale (text, data, bss, ecc.)

Un file ELF descrive cosa deve essere caricato in memoria.

- I **segmenti LOAD** diventano le aree classiche di un processo:
- **Text** → codice del programma (solo lettura + esecuzione)
- **Data** → variabili inizializzate e non inizializzate (lettura + scrittura)
- Altre aree (stack, heap, librerie dinamiche) **non stanno nell'ELF**, le crea il kernel quando avvia il processo.

Fatto il mapping, il loader inizializza il program-counter del processo con lo starting point nell'ELF header e il processo può essere scheduled

Esecuzione e Caricamento

- **gcc** produce file eseguibili nel formato **ELF**
- Il comando **readelf** legge parti di un file elf

- **Header**

- Lista l'header di alto livello nel file ELF

% readelf -h a.out

- **Sections**

- lista le sezioni nell'address space del processo specificate nel file a.out

% readelf -S a.out

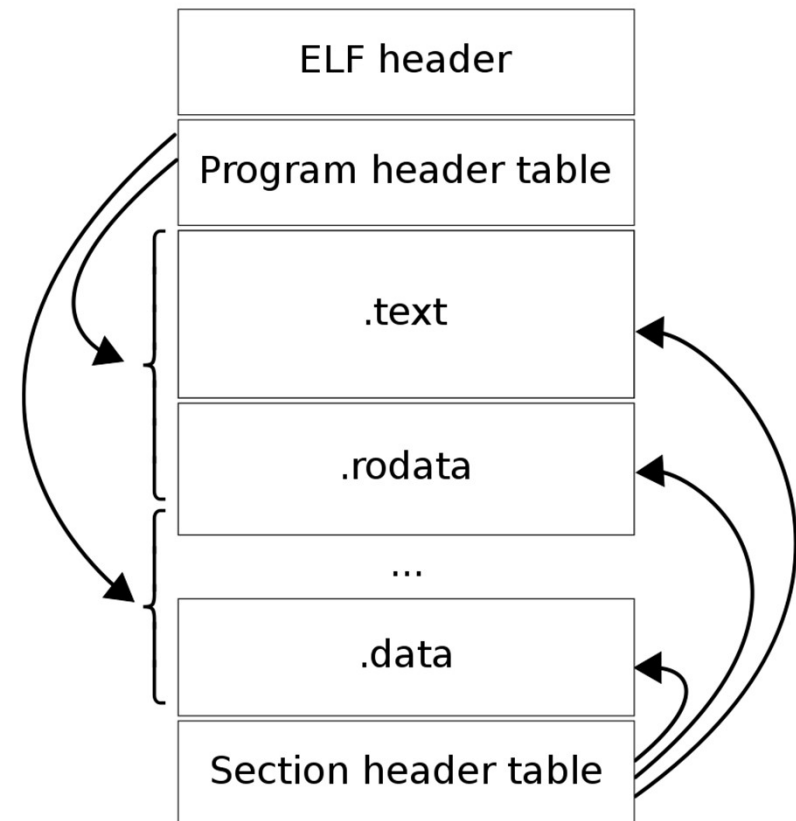
Esecuzione e Caricamento

- **gcc** produce file eseguibili nel formato **ELF**
- Il comando **readelf** legge parti di un file elf
 - **Header**
 - Lista l'header di alto livello nel file ELF

% readelf -h a.out

- **Sections**
 - lista le sezioni nell'address space del processo specificate nel file a.out

% readelf -S a.out



Esecuzione e Caricamento

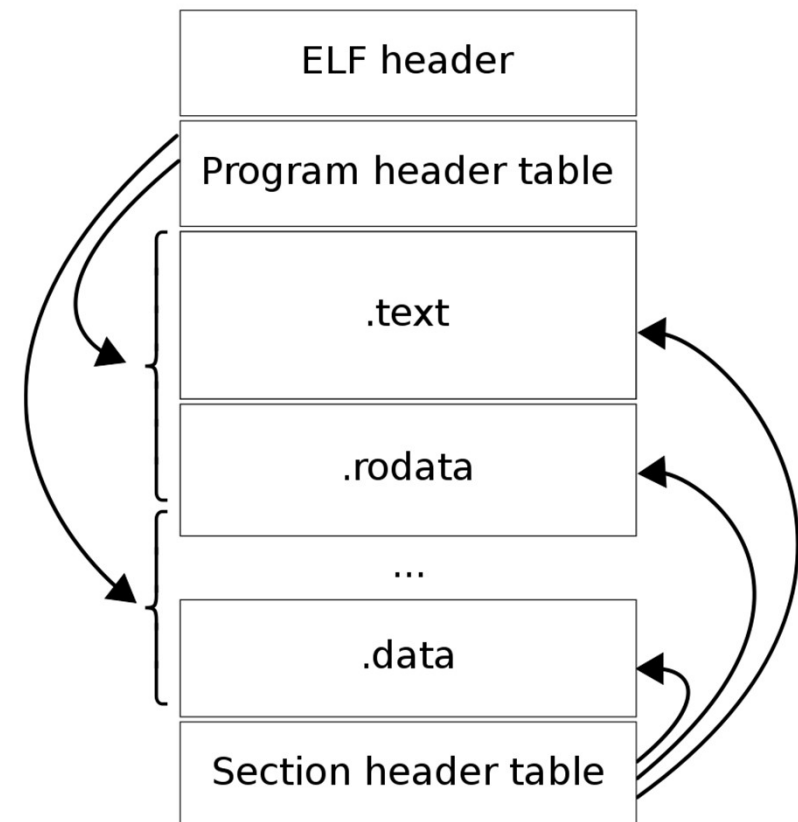
- **gcc** produce file eseguibili nel formato **ELF**
- Il comando **readelf** legge parti di un file elf
 - **Header**
 - Lista l'header di alto livello nel file ELF

% readelf -h a.out

- **Sections**
 - lista le sezioni nell'address space del processo specificate nel file a.out

% readelf -S a.out

- readelf -h nomefile	→ mostra l'intestazione ELF
- readelf -l nomefile	→ mostra i segmenti
- readelf -S nomefile	→ mostra le sezioni
- objdump -d nomefile	→ disassemblato del codice
- ldd nomefile	→ librerie condivise cui dipende



Esecuzione e Caricamento

Tipi di file ELF

- **Relocatable** (es. file `.o`): contengono sezioni, simboli, relazioni, e sono destinati al linking
- **Executable**: ELF che può essere caricato ed eseguito; contiene la **tabella di segmenti** indispensabile per il loader
- **Shared Object** (librerie `.so`): usate per linking dinamico a runtime
- **Core dump**: file ELF che rappresenta lo stato della memoria di un processo terminato in errore

Linking Statico e Dinamico

- Una volta caricato e lanciato, tutto il contenuto del file bin necessario è nello spazio di indirizzamento (virtuale)
- I programmi hanno bisogno anche di librerie
 - Se contenute nell'eseguibile il binario è **staticamente** linkato, ma ogni programma contiene una copia della stessa
- **Linking dinamico** più efficiente in termini di memoria fisica e spazio disco perché carica le librerie una volta sola